

TD n°1 : Expressions régulières

Objectif : concevoir des expressions régulières,
modéliser une réalité imparfaitement connue.

Consigne : pour ces exercices,
ne pas utiliser les opérateurs ? (option) et — (différence ensembliste),
sauf si consigne explicite de les utiliser.

Exercice 1

Définir des expressions régulières qui engendrent les langages suivants :

1. $\Sigma = \{0,1\}$, $\mathcal{L} = \{\omega \in \Sigma^* \mid \text{chaque position impaire contient un } 1\}$, $\varepsilon \notin \mathcal{L}$. NB : la numérotation des positions commence à 1.

Réflexions :

- a. Tout mot d'un langage a des positions impaires (les positions 1, 3, 5, etc.) et des positions paires (les positions 2, 4, 6, etc.).
- b. La spécification semble dire que 1, 11, 10, 111 et 101 sont des mots du langage, mais pas ε , ni 0, ni 01, ni 100.
- c. On peut définir $\mathcal{L}_p$ le langage des sous-mots qui commencent aux positions paires des mots de $\mathcal{L}$, et $\mathcal{L}_i$ le langage des sous-mots qui commencent aux positions impaires des mots de $\mathcal{L}$.
- d. On voit que¹ $\mathcal{L} = \mathcal{L}_i$, $\mathcal{L}_i = \{1\} \bullet (\mathcal{L}_p + \{\varepsilon\})$ et $\mathcal{L}_p = \{0, 1\} \bullet (\mathcal{L}_i + \{\varepsilon\})$.
- e. Et donc $\mathcal{L} = \{1\} \bullet ((\{0, 1\} \bullet (\mathcal{L} + \varepsilon)) + \varepsilon)$.
et²

$$RE_{\mathcal{L}} = 1 \bullet ((0 \mid 1) \bullet 1)^* \bullet (0 \mid 1 \mid \varepsilon).$$

Questions :

- a. Pourquoi '*' ?
- b. Même exercice où une position sur 3 doit être un 1 (en partant de la position 1).
- c. Imaginer une situation concrète où il est important de distinguer les positions paires et les positions impaires.

¹ Ici, on parle de langages, et les notations sont celles des opérations sur les langages.

² Ici, on parle d'expressions régulières, et les notations sont celles des expressions régulières. Le principe différence est qu'un langage $\{a\}$ est décrit par l'expression régulière 'a'. La somme est aussi souvent notée + pour les langages est | pour les expressions régulières. Se rappeler enfin que l'opérateur produit, $\bullet$, est parfois omis. Nous proposons d'utiliser cette facilité avec modération car elle peut nuire à la lisibilité, mais certains outils ont tout simplement supprimé la notation explicite du produit !

Remarques :

- a. Quand les symboles terminaux ne peuvent pas être confondus avec les notations d'expressions régulières, et qu'on travaille hors machine, on peut se dispenser de marquer les symboles terminaux par une notation spéciale (ex. des guillemets).
 - b. Attention ! Quand on répond à la même question sur une machine en utilisant un système dédié, on doit faire rigoureusement comme la documentation du système dédié demande de faire.
 - c. Il vaut souvent le coût de construire une base d'exemples et de contre-exemples. Cela permet d'appuyer les réflexions, de les partager avec ses collègues, de les confronter au client, et de construire des tests unitaires. La spécification peut déjà en contenir, mais ça ne vous dispense pas d'y ajouter les vôtres.
2. $\Sigma = \{a..z, A..Z, '-', 0..9\}$, $\mathcal{L} = \{\text{identificateurs écrits sur } \Sigma, \text{ commençant par une lettre majuscule et terminant par un chiffre}\}$, $\varepsilon \notin \mathcal{L}$.

Réflexions :

- a. La spécification semble dire que A1, Aa1, A11, A-1 sont tous des mots du langage, mais pas A, ni 1, ni 'a', ni toto12, ni univ-rennes1.fr .
- b. Tous les mots commencent par une majuscule, et le langage des majuscules peut être défini par

$$RE_M = (A \mid B \mid C \mid \dots \mid Z).$$
- c. Symétriquement, tous les mots finissent par un chiffre, et on peut définir le langage des chiffres par

$$RE_d = (0 \mid 1 \mid \dots \mid 9).$$
- d. La spécification ne dit rien de ce qui se passe entre les deux. Il faut prendre l'option la plus plausible ; on peut mettre n'importe quelle suite de symboles de Σ . Ce n'importe quoi est Σ^* .
- e. Donc³

$$RE_{\mathcal{L}} = RE_M \bullet \Sigma^* \bullet RE_d.$$

Questions :

- a. Pourquoi univ-rennes1.fr n'appartient-il pas au langage ?
- b. Pourquoi '*' ?
- c. Même exercice où le symbole '-' doit avoir une occurrence au moins.
- d. Même exercice où le symbole '-' doit avoir exactement une occurrence.
- e. Même exercice avec le langage des identificateurs de Python.
- f. Pourquoi le 'd' de RE_d pour l'expression régulière des chiffres ?

³ Attention à la notation Σ^* . Elle n'appartient pas à la notation des expressions régulières. Ici, c'est un raccourci pour RE_{Σ} qu'on aurait définie comme $RE_{\Sigma} = ([a-zA-Z0-9] \mid '-')$.

Remarques :

- a. Souvent, les spécifications expliquent en grand détail les éléments singuliers d'un langage, mais ne disent pas grand chose de la règle générale. C'est à vous de compléter. Mais, sachant que vous travaillez en général pour un tiers, celui qu'on appelle souvent de nos jours le *product-owner*, vous devez évidemment vérifier auprès de celui-ci que vos compléments sont corrects.
 - b. Cela apporte en général beaucoup, en lisibilité, mais aussi en facilité de développement, de concevoir une expression régulière comme un assemblage d'expressions plus simples. Malheureusement, les outils d'expressions régulières ne le peuvent pas toujours. Mais même dans le cas où un outil ne le permet pas, ça vaut le coût de conduire le développement autour d'un assemblage d'expressions, et de rédiger la documentation en les mêmes termes.
3. $\Sigma = \{a..z, '- ', 0..9\}$, $\mathcal{L} = \{\text{identificateurs écrits sur } \Sigma, \text{ commençant par une lettre et ne contenant pas 2 } '- ' \text{ à suivre}\}$, $\varepsilon \notin \mathcal{L}$.

Donner une autre solution si l'opérateur $—$ (différence ensembliste) est autorisé dans les expressions régulières.

Réflexions :

- a. Les consignes négatives doivent absolument attirer l'attention. Commencer par une lettre est une consigne positive, et c'est facile à formaliser avec une expression régulière. Ne pas contenir 2 '- ' à suivre est une consigne négative, et c'est souvent beaucoup plus difficile à formaliser.
- b. C'est typiquement la situation où l'usage de la différence ensembliste serait le bienvenu, mais les outils habituels ne le permettent pas. De plus, la solution avec différence ensembliste ne donne en général aucune intuition de comment faire sans.
- c. Commencer par une lettre est la partie facile. Les mots d'une lettre sont décrits par l'expression régulière $RE_l = (a \mid b \mid \dots \mid z)$, et une expression régulière qui décrit $\mathcal{L}$ aura certainement la forme $RE_l \bullet RE_{\text{suite}}$.
- d. La spécification ne dit pas que les mots de $\mathcal{L}$ contiennent nécessairement un '- ' ou même plusieurs. Elle ne dit pas non plus qu'ils sont forcément suivis d'une lettre ou d'un chiffre.
- e. Les mots 'a', 'a-', aa, aa-, a-a, a-a-, a1, pierre-feuille-ciseau-1515 sont dans le langage, mais pas 1-a ni a1--2, ni a12--.
- f. En première approximation, un '- ' est toujours suivi d'au moins une lettre ou un chiffre, sauf si il n'est suivi de rien. D'où

$$RE_{\text{'-'}} = (\text{'-' } \bullet RE_{dvl}^+ \mid \text{'-' }),$$

où $RE_{dvl} = (RE_d \mid RE_l)$.

- g. Mais il peut y avoir 0, 1 ou plusieurs occurrences de '- ' (vu en d.). D'où

$$RE_{\text{'-'}} = (\text{'-' } \bullet RE_{dvl}^+)^* (\varepsilon \mid \text{'-' }).$$

- h. Mais un mot du langage doit aussi commencer par une lettre (vu en c.), et peut contenir autant de lettres et de chiffres qu'on veut avant le premier '-'. D'où

$$RE_{\mathcal{L}} = RE_I \bullet RE_{dvl}^* \bullet RE_{',-}.$$

- i. Et avec la différence ensembliste ? On aurait⁴

$$\text{RE}_{\mathcal{L}} = \text{RE}_I \bullet (\Sigma^* - (\Sigma^* \bullet ' - ' \bullet ' - ' \bullet \Sigma^*)).$$

Questions :

- Expliquer les '+' et '*'.
- Expanser les expressions régulières intermédiaires pour se rendre compte de combien ça devient illisible.

Remarques :

- a. La difficulté de modéliser les consignes négatives est une observation très générale. Voir par exemple la modélisation en base de données.
- b. Beaucoup de mauvaises solutions viennent de réflexions trop superficielles où on corrige localement une anomalie constatée en un point sans réfléchir à ce que ça donne globalement.
- c. Il arrive que des expressions régulières soient vraiment complexes par nécessité, mais quand même, la plupart du temps, une expression régulière complexe ne l'est que par confusion des esprits.
- d. Se rappeler que, sur papier, on peut omettre les •. On peut donc aussi bien écrire

$$RE_c = RE_l + RE_d + RE_{d'} + RE_{d''}$$

ou

$$RE_f = RE_I (\Sigma^* - (\Sigma^* \text{ ' - ' } \Sigma^*)).$$

Mais attention, les outils sont moins complaisants !

4. Le langage des expressions d'heure (par exemple, 13h42, mais pas 27h72) : un ou deux chiffres dénotant l'heure, suivi par la lettre 'h', suivi par deux chiffres dénotant les minutes.

L'heure doit être entre 0 et 23, et les minutes entre 00 et 59.

Les chiffres dénotant l'heure ne doivent pas commencer par un 0 (3h42, mais pas 03h42) sauf pour 0 heure (exemple, 0h15).

On utilisera les notations d'intervalles de symboles terminaux. Ex. $[3 - 6]$ signifie $(3 \mid 4 \mid 5 \mid 6)$. Se rappeler que $[135]$ signifie $(1 \mid 3 \mid 5)$.

Réflexions :

- a. Clairement, les champs heures et minutes suivent des lois différentes.
On aura donc

$$RE_f = RE_h \text{ 'h' } RE_m$$

et on s'occupera séparément de RE_h et de RE_m .

⁴ Même remarque que pour la note 3 plus haut. Et faire attention à la différence entre le tiret du langage, '-', et celui du méta-langage '—'.

- b. En plus des exemples et contre-exemples fournis, on peut noter que 0h00, 1h21, 9h21, 19h21 et 21h19 sont des notations légales, mais pas 25h19, ni 00h0.
- c. Si les heures commencent par 0 ou un chiffre plus grand que 2 (strictement), il n'y a rien après. Si elles commencent par 1 on peut trouver après un chiffre quelconque ou rien, et si elles commencent par 2 on peut trouver après un chiffre entre 0 et 3 ou rien.
- d. En conséquence,

$$RE_h = (0 \mid [3 - 9] \mid 1 [0 - 9]^? \mid 2 [0 - 3]^?).$$
- e. Un raisonnement similaire donne

$$RE_m = [0 - 5] [0 - 9].$$

Questions :

- a. Pourquoi ne surtout pas écrire $RE_h = [0 - 23]$ et $RE_m = [00 - 59]$?
- b. Aux États-Unis, la numérotation des étages commence à 1, et il n'y a généralement pas de 13^{ème} étage. Donner le langage des numéros d'étage pour les immeubles de moins de 135 étages.

Remarques :

- a. On utilise ici le quantificateur '?' qui signale qu'un composant est optionnel : $RE^? = (RE \mid \varepsilon)$. Beaucoup d'outils proposent une notation de ce genre.
 - b. Les espaces dans les notations d'intervalles, ex. $[0 - 3]$, ne sont là que pour aérer. Faire attention qu'un outil pourrait avoir une autre idée sur la question et donner un sens particulier au caractère espace.
5. Le langage des immatriculations françaises (par exemple, AZ-229-BC).
 Ces numéros sont composés de 9 signes, suivant le format suivant : deux lettres, un tiret, trois chiffres, un tiret et deux lettres.
 Les lettres I, O et U sont exclues.
 Les combinaisons SS et WW à gauche et SS à droite sont exclues aussi.

Donner une autre solution si l'opérateur $-$ (différence ensembliste) est autorisé dans les expressions régulières.

Réflexions :

- a. À nouveau, on ressent cruellement le manque d'une notation de différence ensembliste. Il va falloir s'armer de courage.
- b. Comme pour les heures, on va identifier des sous-langages indépendants :

$$RE_{\text{L}} = RE_1 \text{ '-' } RE_2 \text{ '-' } RE_3.$$

- c. RE_2 ne pose pas spécialement de problème :

$$RE_2 = [0 - 9]^3.$$

- d. Pour RE_1 et RE_3 tout se complique. D'abord l'alphabet : toutes les lettres sauf I, O et U !

$$RE_{\text{immat}} = [A - H] \mid [J - N] \mid [P - T] \mid [V - Z].$$

- e. Regardons RE_3 . Un S en première position doit nous alerter car il ne peut pas être suivi d'un autre S. Il faut donc distinguer les S des autres lettres permises. D'où,

$$RE_{\text{limmat-sauf-S}} = [A - H] \mid [J - N] \mid [P - R] \mid [T] \mid [V - Z]$$

et⁵

$$RE_3 = (S RE_{\text{limmat-sauf-S}} \mid RE_{\text{limmat-sauf-S}} RE_{\text{limmat}})$$

- f. C'est la même chose pour RE_1 , en un peu plus compliqué.

$$RE_1 = (S RE_{\text{limmat-sauf-S}} \mid W RE_{\text{limmat-sauf-W}} \mid RE_{\text{limmat-sauf-S-ou-W}} RE_{\text{limmat}}),$$

avec

$$RE_{\text{limmat-sauf-W}} = [A - H] \mid [J - N] \mid [P - T] \mid [V] \mid [X - Z]$$

et

$$RE_{\text{limmat-sauf-S-ou-W}} = [A - H] \mid [J - N] \mid [P - R] \mid [T] \mid [V] \mid [X - Z].$$

Questions :

- Pourquoi toutes ces restrictions ? Pas de I, de O ou de U ? Pas de combinaison SS ou WW ?
- Proposer une solution qui utilise la différence ensembliste.

Remarques :

- On a utilisé une notation d'exposant $[0 - 9]^3$ pour signifier $[0 - 9] [0 - 9] [0 - 9]$. Ici, le gain est faible, mais imaginez de devoir modéliser les numéros INSEE. Beaucoup d'outils proposent une notation de ce genre. En général, ces outils permettent des variations comme $[0 - 9]^{3,7}$ pour signifier entre 3 et 7 chiffres.
- Certains outils qui ne permettent pas de noter de différence ensembliste entre expressions régulières en général, permettent de noter des différences ensemblistes entre intervalles de symboles terminaux. On écrirait alors

$$RE_{\text{l-immat}} = [A - Z] - [IOU]$$

ou

$$RE_{\text{l-immat-sauf-S-ou-W}} = RE_{\text{l-immat}} - [SW].$$

Ce qui est quand même bien plus commode⁶.

- Bien réfléchir à la condition exprimée en début de la remarque b.

6. Le langage des URL : par exemple

`https://foad.univ-rennes1.fr/course/view.php`

ou `https://foad.univ-rennes1.fr/course/157`

ou `ftp://ftp.ccsd.cnrs.fr`

ou `ftp://ftp.ccsd.cnrs.fr:80`.

⁵ Faire attention ici à la priorité des opérateurs. On convient généralement que $(r s \mid t u)$, ou $(r \bullet s \mid t \bullet u)$, doit se lire $((r \bullet s) \mid (t \bullet u))$ et pas $(r \bullet (s \mid t) \bullet u)$. L'opérateur $\bullet$ est donc plus prioritaire que l'opérateur $\mid$. Tout comme en arithmétique élémentaire le produit est prioritaire sur la somme. Attention quand même en utilisant un outil particulier ; il faudra lire la documentation pour savoir qui est prioritaire sur qui

⁶ Noter quand même que cet exercice fait cohabiter trois sortes de tiret ! Le tiret du langage objet, '-', celui des notations d'intervalle de symboles, -, et celui de la notation de différence ensembliste, —. Il faut donc être vigilant.

Réflexions :

- a. Le langage des URL est complexe ! Il faut vraiment y aller progressivement.
- b. Le langage complexe des URL se décompose certainement en des sous-langages plus simples qui traitent chacun d'une partie de la sémantique.
- c. Que sait-on de la sémantique des URL ? Un premier segment, jusqu'au '://' identifie le protocole utilisé : `http` pour le web, `https` pour le web sécurisé, `ftp` pour l'échange de fichier, `file` juste pour lire un fichier, etc. Le segment d'après, jusqu'au premier '/', désigne un service via son adresse. Enfin, un troisième segment, jusqu'à la fin, s'interprète comment une requête dont un chemin dans un système de fichier est un cas particulier. D'où

$$RE_{url} = RE_{protocole} : // RE_{service} (/ RE_{query})^?.$$

Chaque segment suscite de nombreuses questions !

- d. Le protocole ? Vaut-il mieux énumérer tous les protocoles qu'on imagine, ex. $RE_{protocole} = (http | https | ftp | file | ssh | mailto | \dots)$, ou plutôt formaliser un langage attrape-tout, à charge pour un post-traitement de filtrer les protocoles d'intérêt pour une application donnée, ex. $RE_{protocole} = RE_l^+ ?$

Il faut sans doute mieux préférer la seconde solution, qui sera plus générique.

- e. Le service ? Ne pas oublier que la désignation du service peut aller jusqu'à la spécification d'un numéro de port, ex. `:80`. Ne pas oublier non plus qu'elle peut se faire via une adresse IP, ex. `ou 127.0.0.1`, ou même via des identificateurs conventionnels, ex. `localhost`. On peut donc proposer

$$RE_{service} = (RE_{IP} | RE_{domain} | RE_{symbole}) ':' RE_{port}.$$

- f. On peut poser

$$RE_{IP} = (RE_{octet} '.')^3 RE_{octet}, \text{ où } RE_{octet} = [0 - 9]^{1,3}$$

si on ne cherche pas à vérifier que les champs de l'adresse doivent être compris entre 0 et 255, ou

$$RE_{octet} = (2 5 [0 - 5] | 2 [0 - 4] [0 - 9] | [0 1] [0 - 9]^? [0 - 9]^?)$$

si on le vérifie. On reconnaît le style de modélisation employé pour les heures.

- g. On peut poser

$$RE_{domain} = (RE_{name} '.')^+ RE_l^{2,}$$

avec

$$RE_{name} = [a - z 0 - 9]^+ ('-' [a - z 0 - 9]^+)^*.$$

Cependant, cette expression régulière ne vérifie pas que les noms de domaine font moins de 63 caractères. Elle ne vérifie pas non plus que les TLD (*top-level domain*, comme `.ci` ou `.fr`) sont valides.

- h. La situation de RE_{query} est encore plus complexe puisqu'il s'y mélange des considérations de style de notation de chemin dans un système de fichiers (avec des '/' comme en UNIX/Linux, ou avec des '\' comme en Windows), de compatibilité de jeux de caractères entre les protocoles réseau (ex. `http`) et les systèmes de fichiers, de style de requêtage (ex. requête REST), de protocole `http`, et de devoir accepter dans les requêtes des chaînes fournies par l'utilisateur du côté client.

- i. Il n'y a donc pas de réponse complètement définitive à cette question.

Questions :

- a. Même question avec les numéros INSEE.

Remarques :

- a. Se rappeler que la syntaxe préfigure la sémantique. Cela aide souvent à décomposer proprement un langage complexe en sous-langages moins complexes.
- b. Toujours être modeste face à la modélisation du monde réel. Il y a deux approches modestes. La première consiste à renoncer à modéliser tous les détails. Dans ce cas, le plus souvent il vaut mieux sur-générer que sous-générer. La seconde consiste à se demander si il n'existe pas un package qui ferait déjà le travail ! Selon la source du package, on peut avoir plus ou moins confiance en la modélisation qui est faite.
- c. On peut trouver sur le web de très nombreuses propositions d'expressions régulières pour décrire et reconnaître le langage des URL. Il ne faut surtout pas les copier bêtement, c-à-d. sans tenter de comprendre ce qu'elles modélisent et si ça convient à son besoin. Mais elles peuvent servir de base.
- d. Mais ça veut dire qu'on est capable de lire une expression régulière écrite par un autre.

Exercice 2

Lire attentivement le programme suivant et en déduire le langage de ses unités lexicales.

Pour cela, découper le programme en unités lexicales, puis définir les expressions régulières qui les engendrent.

```
function reverse:
read X
%
    Y := nil ;
    while X do X, Y := (tl X), (cons(hd X) Y) od
%
write Y

function nba:
read X
%
    Nba := nil ;
    while X do
        if (hd X) ?= a then Nba := (cons nil Nba) fi
    od
%
write Nba
```


Réflexions :

- a. Se rappeler le principe de « double articulation » des langages. Dès qu'un langage (naturel ou artificiel) est un peu sophistiqué, il articule des symboles élémentaires (ex. des lettres ou des phonèmes) pour former un vocabulaire intermédiaire (ex. des mots-lexicaux, communément appelés unités lexicales), et il articule les symboles de ce vocabulaire intermédiaire pour former des structures de plus haut niveau (ex. des mots-phrases ou des mots-programmes).
- b. Partant de l'idée que les structures de plus haut niveau préfigurent le sens global du document, les unités lexicales sont des unités de sens non décomposables. Par exemple, `function` semble introduire des définitions, comme `def` en Python ou `class` en Java. Le décomposer en `func+tion`, par imitation de `add+tion` ou `subtrac+tion`, peut être tentant, mais l'échantillon ne montre pas d'utilisation autonome de `tion` qui lui donnerait un sens informatique.
- c. Enfin, la culture informatique doit servir. On sait que les parenthèses sont souvent utilisées pour délimiter des sous-structures. On sait que le signe `';` sert souvent pour former des suites d'opérations à réaliser en séquence. On sait que `:=` est une notation possible pour l'affectation. On sait que le signe `' '` (espace) joue souvent un rôle de séparateur. Chacun de ces savoirs connaît des contre-exemples, mais ils peuvent guider l'exploration. On sait que les notations d'affectation visent à distinguer la notation de la mémoire dans laquelle on écrit de la notation de la valeur qu'on écrit dedans. On sait aussi qu'on appelle souvent « partie gauche » ou *lvalue* la première justement parce qu'on l'écrit à gauche du signe d'affectation. Et on sait que très souvent la partie gauche est un identificateur de variable. On peut pousser ce genre de raisonnement tant qu'on n'est pas contredit.
- d. On en déduit que `:=` semble être un signe d'affectation, et que `X`, `Y` et `Nba` semblent être des notations de variable.
- e. À l'inverse, on voit que `reverse`, `nba` ou `cons` ne sont jamais utilisés en partie gauche d'une affectation. Il est donc plausible que seuls des identificateurs capitalisés puissent dénoter des variables.
- f. Au total, on reconnaît des ponctuations comme `':'`, `','`, `'%'` et `' '`, des mots clés comme `if`, `fi`, `do`, `od`, `function`, `read`, `write`, `cons`, `hd` et `tl`, et des identificateurs de variables comme `X`, `Y` et `Nba`, qui semblent avoir en commun d'être capitalisés. D'où

$$RE_{\text{punctuation}} = (: | ; | \% | , | \dots)$$

$$RE_{\text{keyword}} = (\text{if} | \text{fi} | \text{do} | \text{od} | \dots)$$

$$RE_{\text{idvar}} = RE_M RE_l^*$$

- g. Le 'a' de « `(hd X) ?= a` » a un statut assez incertain et plus d'exemples pourraient mieux éclairer son cas. En l'état, et comme cette notation est précédé d'un `if`, on peut penser qu'il s'agit de la condition d'une instruction conditionnelle. Si c'est le cas, `'?='` est sans doute un opérateur de comparaison, et alors 'a' serait une notation de constante, à quoi on comparerait le résultat de `(hd X)`. D'où

$$RE_{\text{idconst}} = RE_m RE_l^* .$$

Questions :

- a. Que pourrait-on dire de plus au sujet de `read`, `write` et `'%'` ?
- b. Que pourrait-on dire des variables `X` et `Y` de la fonction `reverse` ?

Remarques :

- a. La question de où situer le niveau lexical n'est pas simple. Elle est assez similaire à celle des valeurs atomiques dans la définition de la 1^{ère} forme normale des bases de données relationnelles. Par exemple, une notation décimale comme 3,14 peut évidemment être décomposée en une partie entière et une partie décimale. Pour autant, peut-on dire qu'une notation décimale n'est pas atomique ? Pareillement, des noms de *getters* et *setters* comme `get_date` et `set_date` forment-ils des unités lexicales atomiques, ou doit-on les analyser comme des combinaisons de `get`, `set` et `date` ?
- b. Tout ça peut passer pour une réflexion très éthérée, mais c'est pourtant une question fondamentale pour qui construit des modèles.
- c. Savoir analyser superficiellement des notations, même d'un langage qu'on ne connaît pas, et une compétence extrêmement utile car on est souvent confronté à programmer par imitation.
- d. Dans un livre très classique, « *The Pragmatic Programmer* » (disponible à la BU), Andy Hunt et Dave Thomas recommandent que les programmeurs apprennent un langage de programmation par an. Ils sont entre autres les concepteurs de Junit pour le test unitaire, et des spécialistes de l'agilité ou de Ruby. Ils savent de quoi ils parlent.
- e. En programmation, passer de l'état où on déclare connaître Java plus Python parce qu'on les a appris à l'école à l'état où on ne se sent pas démunie de travailler dans un langage qu'on n'a jamais étudié, est comme passer de l'adolescence à l'âge adulte. Ça doit être votre objectif !

Exercice 3 (Préparation du TP à faire chez soi)

Se familiariser avec la bibliothèque *Python regular expressions*, `regex`, en lisant notamment les pages de tutoriel proposées sur l'ENT (dans Moodle) et en faisant des tests. On notera au passage que la syntaxe utilisée ici pour les RegEx est un peu différente de la syntaxe formelle utilisée en cours.

Identifier les étapes principales de l'analyse d'un texte par une expression régulière (quelles classes et fonctions Python ?).

Écrire un programme Python effectuant un test élémentaire (sans lecture de fichier, ni lecture au clavier) : analyse d'une chaîne de caractère avec une expression régulière simple.

Tester les expressions régulières de ce TD.